

Phase 7A Approved AI Source Reference

Complete consolidated conclusion of the approved Phase 7A HRMS module documents, written for AI retrieval and human review.

Covers approved delivery scope, architecture, database model, APIs, events, user operations, examples, controls, and implementation guardrails.



Web visual: Wikimedia Commons team meeting photo, used as a collaboration visual.

Document role	AI source reference and stakeholder reference
Status basis	Summarizes approved Phase 7A documentation
Module packs covered	15 module packs x 5 approved docs = 75 module documents
Shared hardening covered	5 approved standards/backlog documents
Language style	Simple, explicit, searchable, and implementation-oriented

1. Executive Conclusion

Phase 7A is the first development phase that turns the HRMS platform from approved design into a working enterprise foundation. It is not only a set of HR screens. It delivers the platform base required for a configurable, multi-tenant, auditable HRMS, and then builds the first business modules on top of that base.

The approved outcome is: build Tenant Catalog and RLS, Branch / Office Hierarchy, Identity and Access, Effective Dating, Audit and Time Machine, Event Bus and Outbox, Rule Engine, Workflow Studio, Configuration-as-Data, Core HR and ESS, Leave, Attendance, Shift Foundation, Payroll with India compliance foundation, and Standard Reports.

This PDF is intentionally more detailed than the presentation brief. It is suitable as an AI source because each module section uses consistent labels: conclusion, approved delivery, architecture, database, API, events, UI operation, example, and test focus.

Canonical rule: if this PDF conflicts with an approved source document, the approved source document remains the legal design authority. This PDF is a consolidated reference, not a replacement for the source files.

2. Who Phase 7A Serves

User group	What Phase 7A gives them
Employees	Self-service profile changes, leave requests, attendance visibility, payslips, and request status.
Managers	Team approvals, leave and attendance decisions, shift-aware attendance visibility, team information, and task inbox.
HR admins	Employee master data, policies, workflows, configuration, calendars, reports, and evidence.
Payroll teams	Payroll setup, dry runs, exception review, approvals, publish, payslips, and compliance evidence.
Tenant admins	Tenant setup, branch/office hierarchy, feature flags, provider configuration, branding, security, and controlled changes.
Auditors and security teams	Audit trails, time-machine reconstruction, access evidence, exports, and investigation support.

3. Phase 7A Development Order

Sprint	Theme	Reason
S1	Tenant + Identity + Branch scope	No business module can be safely built without tenant context, RLS, branch scope, authentication, RBAC, and ABAC.
S2	Effective Dating + Audit + Events	History, evidence, and event reliability must exist before modules create important business records.
S3	Rule Engine + Workflow + Configuration	Business behavior must be configurable and approval-driven before Leave, Attendance, and Payroll depend on it.
S4	Core HR + ESS	Employee master data, branch assignments, and job assignments are required by downstream modules.
S5	Leave	First complete HR workflow using rules, workflow, balances, payroll impact, and audit.
S6	Attendance + Shift Foundation + first connector	Shift-aware time input and summaries are needed for reliable payroll.
S7	Payroll engine	Calculates payroll using Core HR, Leave, Attendance, rules, and snapshots.
S8	India compliance foundation	Adds statutory packs and compliance outputs on top of payroll engine.
S9	Workflow hardening	Strengthens delegation, SLA, escalation, migration, and process resilience.
S10	Reports + release hardening	Delivers standard reports, exports, performance, security, and release readiness.

4. Approved Source Document Inventory

Each Phase 7A module pack has five approved documents: Business Requirements, Technical Design, Database Design, UI Design, and Test Plan. The PDF summarizes their conclusions; the paths below identify the source documents used.

Module	Business	Technical	Database	UI	Test
Tenant Catalog + Row-Level Security	FEAT-TENANT-001	TECH-TENANT-001	DB-DESIGN-TENANT-001	UI-TENANT-001	TEST-TENANT-001
Branch / Office Hierarchy and Scoped Administration	FEAT-BRANCH-001	TECH-BRANCH-001	DB-DESIGN-BRANCH-001	UI-BRANCH-001	TEST-BRANCH-001
Identity, Authentication, RBAC, and ABAC	FEAT-IDENTITY-001	TECH-IDENTITY-001	DB-DESIGN-IDENTITY-001	UI-IDENTITY-001	TEST-IDENTITY-001
Effective Dating and Bitemporal Core	FEAT-EFFECTIVE-DATING-001	TECH-EFFECTIVE-DATING-001	DB-DESIGN-EFFECTIVE-DATING-001	UI-EFFECTIVE-DATING-001	TEST-EFFECTIVE-DATING-001
Audit and Time Machine	FEAT-AUDIT-001	TECH-AUDIT-001	DB-DESIGN-AUDIT-001	UI-AUDIT-001	TEST-AUDIT-001
Event Bus and Transactional Outbox	FEAT-EVENT-BUS-001	TECH-EVENT-BUS-001	DB-DESIGN-EVENT-BUS-001	UI-EVENT-BUS-001	TEST-EVENT-BUS-001
Rule Engine	FEAT-RULE-ENGINE-001	TECH-RULE-ENGINE-001	DB-DESIGN-RULE-ENGINE-001	UI-RULE-ENGINE-001	TEST-RULE-ENGINE-001
Workflow Studio	FEAT-WORKFLOW-001	TECH-WORKFLOW-001	DB-DESIGN-WORKFLOW-001	UI-WORKFLOW-001	TEST-WORKFLOW-001
Configuration-as-Data	FEAT-CONFIGURATION-001	TECH-CONFIGURATION-001	DB-DESIGN-CONFIGURATION-001	UI-CONFIGURATION-001	TEST-CONFIGURATION-001
Core HR and Employee Self-Service	FEAT-CORE-HR-001	TECH-CORE-HR-001	DB-DESIGN-CORE-HR-001	UI-CORE-HR-001	TEST-CORE-HR-001
Leave Management	FEAT-LEAVE-001	TECH-LEAVE-001	DB-DESIGN-LEAVE-001	UI-LEAVE-001	TEST-LEAVE-001
Attendance and First Connector	FEAT-ATTENDANCE-001	TECH-ATTENDANCE-001	DB-DESIGN-ATTENDANCE-001	UI-ATTENDANCE-001	TEST-ATTENDANCE-001
Shift Foundation	FEAT-SHIFT-FOUNDATION-001	TECH-SHIFT-FOUNDATION-001	DB-DESIGN-SHIFT-FOUNDATION-001	UI-SHIFT-FOUNDATION-001	TEST-SHIFT-FOUNDATION-001
Payroll and India Compliance	FEAT-PAYROLL-001	TECH-PAYROLL-001	DB-DESIGN-PAYROLL-001	UI-PAYROLL-001	TEST-PAYROLL-001
Standard Reports	FEAT-REPORTING-001	TECH-REPORTING-001	DB-DESIGN-REPORTING-001	UI-REPORTING-001	TEST-REPORTING-001

Shared hardening documents approved for Phase 7A:

Document	Path	Purpose
API, Event, NFR, and Runbook Standard	docs/09-development/PHASE-7A-STD-001-api-event-nfr-runbooks.md	Every module must define versioned APIs, event contracts, error responses, non-functional targets, observability, and runbook expectations.
Database Classification, Migration, and Rollback Standard	docs/06-database/DB-DESIGN-PHASE-7A-STD-001-data-classification-migration.md	Every database design must classify data, protect PII/payroll data, define migrations, rollback, verification, retention, and legal-hold behavior.
UI Accessibility and States Standard	docs/07-ui-ux/UI-PHASE-7A-STD-001-accessibility-states.md	Every UI must handle loading, empty, error, permission-denied, approval, audit, responsive, and accessibility states.
Testing Traceability and Abuse Standard	docs/10-testing/TEST-PHASE-7A-STD-001-traceability-abuse.md	Tests must trace requirements to scenarios and include tenant isolation, abuse, negative, security, performance, and recovery cases.

Document	Path	Purpose
Phase 7A Hardening Backlog	docs/21-product-backlog/PHASE-7A-HARDENING-BACKLOG-001-review-recommendations.md	Module-specific hardening items are tracked without blocking the approval gate, but they must be considered during implementation planning.

5. Platform Architecture Conclusion



Web visual: Wikimedia Commons server infrastructure photo, used as a platform architecture visual.

The approved architecture is a modular, configurable, event-aware HRMS platform. The backend is .NET-first, the frontend is React/Next.js/TypeScript, the primary database is SQL Server, cache is Redis, messaging is RabbitMQ first with Azure Service Bus optional later, storage uses Azure Blob Storage, search uses Elasticsearch, and OpenAPI documentation is mandatory for every API.

5.1 Simple Architecture View

```
User / Browser / API Client
-> Next.js / React UI and API Gateway
-> .NET Application Services and Module APIs
-> Shared Platform Services
    Tenant Context + RLS
    Identity + RBAC + ABAC
    Effective Dating
    Audit + Time Machine
    Event Bus + Outbox
    Rule Engine
    Workflow Studio
    Configuration-as-Data
-> Business Modules
    Core HR, Branch/Office, Leave, Attendance, Shift Foundation, Payroll, Reports
-> SQL Server, Redis, RabbitMQ, Blob Storage, Search, Observability
```

5.2 Architectural Principles Approved

- Tenant isolation is mandatory at request, service, database, cache, event, storage, search, job, report, and telemetry levels.
- Customer customization uses configuration, feature flags, extensions, plugins, provider adapters, and manifests; core logic must not be changed per customer.
- Business rules are stored and versioned in the Rule Engine or configuration; no hardcoded policy thresholds in product code.
- Workflow is a platform service used by many modules, not a feature hidden inside Leave or Payroll.
- Published rules, workflow definitions, payroll runs, and high-risk configuration versions are immutable.
- Effective dating and time-machine capability are required from the beginning because adding them later is costly and risky.
- Every module must expose versioned OpenAPI contracts, support RBAC/ABAC, create audit records, and participate in test traceability.

5.3 Plug-and-Play Extension Model

Future modules can be ingested by registering a module manifest, defining API and event contracts, adding tenant-scoped schema tables, using shared authorization/audit/effective-dating services, and publishing UI extension slots. Existing core services should be extended through interfaces and configuration rather than edited for a customer-specific requirement.

6. Database Architecture Conclusion

The database design is SQL Server-first and enterprise-oriented. It uses schemas to separate platform foundations and business modules. Every tenant-scoped table includes TenantId and is protected by RLS. Important time-changing facts use effective dating and, where approved, SQL Server temporal history. Published business results are immutable or corrected through controlled adjustment records.

6.1 Approved Schema Map

Schema	Purpose
catalog	Tenant catalog, placement, feature flags, branding, provider config, provider health.
security	Users, roles, permissions, ABAC policies, branch scopes, sessions, delegation, impersonation, break glass.
org	Branch and office hierarchy, hierarchy closure, employee branch assignment, scoped administration evidence.
core	Effective-dated entity registration, change requests, conflicts, shared platform facts.
audit	Audit records, field changes, security events, time-machine snapshots, export requests.
integration	Event contracts, outbox, inbox, dead letters, replay requests.
rules	Rule sets, rule versions, decision tables, simulations, evaluation logs.
workflow	Workflow definitions, versions, instances, tasks, decisions, SLA events.
config	Module manifests, schemas, configuration items/versions, feature flags, import/export.
hr	Person, employee, assignment, org units, locations, ESS requests, document metadata.
leave	Leave types, policies, calendars, requests, transactions, balance projections.
attendance	Attendance policies, shifts, shift assignments, devices, raw punches, day summaries, regularization, sync runs.
payroll	Calendars, periods, components, structures, salary assignments, runs, components, payslips.
reporting	Report definitions, runs, exports, employee and payroll projections.

6.2 Tenant Isolation Example

Request resolves TenantId on the server.
Database connection sets SESSION_CONTEXT('TenantId').
Every tenant table has TenantId.
RLS FILTER predicate hides other tenants' rows.
RLS BLOCK predicate prevents wrong-tenant inserts/updates.
EF query filters and service checks are additional defense, not replacement.

6.3 Effective Dating Example

```
hr.EmployeeAssignment
  EmployeeId = E100
  Department = Sales
  EffectiveFrom = 2026-01-01
  EffectiveTo = 2026-06-30
```

```
hr.EmployeeAssignment
  EmployeeId = E100
  Department = Finance
  EffectiveFrom = 2026-07-01
  EffectiveTo = NULL
```

As-of 2026-06-30 -> Sales
As-of 2026-07-01 -> Finance
History explains approval, reason, old row, new row, and audit trail.

6.4 Leave Ledger Example

Leave balance is not overwritten.
Opening balance transaction: +12.0
Approved leave request: -3.0
Cancellation or correction: +3.0 or adjustment transaction
Projection = sum of ledger transactions by employee, leave type, and as-of date.
If projection is wrong, rebuild it from leave.LeaveTransaction.

6.5 Payroll Reproducibility Example

PayrollRun stores:

PayPeriodId
SourceSnapshotHash
RuleVersionSnapshotJson
Status and approval evidence

PayrollRunEmployee stores employee totals.
PayrollRunComponent stores exact component amounts.
Published payroll is immutable.
Corrections use adjustment runs or controlled correction records.

7. Common Operating Model

Phase 7A uses a repeatable business pattern so users see simple actions while the platform enforces governance behind the scenes.

Configure -> Request -> Validate -> Approve -> Calculate -> Publish -> Audit

Configure: admin publishes tenant policy, form, rule, workflow, feature, or provider settings.

Request: employee, manager, HR, payroll, connector, or job submits a business action.

Validate: tenant, RBAC, ABAC, rule engine, dates, schema, and idempotency are checked.

Approve: workflow routes tasks, delegation, SLA, and maker-checker decisions.

Calculate: leave balance, attendance summary, payroll result, or report projection is produced.

Publish: approved output becomes visible or exportable.

Audit: every important step is traceable with tenant and correlation id.

7.1 End-to-End Example: Leave to Payroll

- Employee submits leave request from UI.
- Tenant, identity, role, ABAC, leave policy, day-count rules, and calendar are validated.
- Leave ledger reserves requested days and workflow assigns approval task.
- Manager approves; leave ledger debits balance and LeaveApproved event is written to outbox.
- Payroll consumes the approved event and treats unpaid or loss-of-pay impact during the payroll source snapshot.
- Reports can show leave history and payroll impact with audit evidence.

7.2 End-to-End Example: Attendance to Payroll

- Connector imports raw punches from a configured device.
- Duplicate punch detection prevents double counting.
- Attendance engine resolves the employee's effective shift and calculates day summary using tenant rules.
- Employee regularization requests follow workflow if correction is needed.
- Approved day summary produces payroll-impact event.
- Payroll snapshots shift-aware attendance data at cutoff and can explain late/absence impact.

8. Tenant Catalog + Row-Level Security

Phase: S1 foundation | Schema: catalog plus security RLS | Approved source key: TENANT

Approved conclusion

This module is the foundation of the SaaS platform. It defines every customer, their status, placement, feature entitlements, branding, provider choices, and the tenant context required before any business data can be accessed. The approved design makes tenant isolation a platform rule, not a developer preference.

Approved delivery

- Tenant catalog with tenant, shard, branding, feature flag, provider, provider config, and provider health records.
- Server-side tenant resolution from host, token-safe hints, or platform-owner context.
- SQL Server Row-Level Security for every tenant-scoped table.
- Tenant lifecycle controls for provision, activate, suspend, offboard, and placement change.
- Tenant-safe namespaces for cache, search, storage, events, logs, telemetry, and providers.

Architecture and dependencies

- The request pipeline resolves tenant context before application services run.
- Application services receive an immutable tenant context; repositories cannot run without it.
- Entitlements are checked across API, UI, jobs, reports, integrations, events, and AI.
- Provider abstraction reads tenant provider configuration without exposing secrets to business code.

Database model

Key tables / records
catalog.Tenant
catalog.TenantShard
catalog.TenantBranding
catalog.TenantFeatureFlag
catalog.TenantConfigVersion
catalog.ProviderType
catalog.Provider
catalog.TenantProviderConfig
catalog.ProviderHealth
security.fn_tenant_predicate
security.TenantIsolation policy

API and OpenAPI examples

- GET /api/v1/tenants/{id}
- POST /api/v1/tenants
- POST /api/v1/tenants/{id}/suspend
- GET /api/v1/tenants/{id}/features
- PUT /api/v1/tenants/{id}/providers/{providerType}

Event examples

- TenantProvisioned
- TenantActivated
- TenantSuspended

- TenantOffboardingStarted
- TenantFeatureFlagsChanged
- TenantProviderConfigChanged

Example scenario

Tenant A and Tenant B can both use the same People table. Every row has TenantId. When Tenant A's request reaches SQL Server, SESSION_CONTEXT contains Tenant A's id, so RLS hides Tenant B rows and blocks wrong-tenant writes even if a query is written incorrectly.

Test and quality focus

- Cross-tenant reads and writes are blocked at service, EF filter, and SQL RLS layers.
- Suspended tenants cannot use APIs, jobs, reports, events, or integrations.
- Provider config and tenant lifecycle changes are audited and invalidate caches.

Source document paths

- docs/02-product-requirements/FEAT-TENANT-001-business-requirements.md
- docs/09-development/TECH-TENANT-001-technical-design.md
- docs/06-database/DB-DESIGN-TENANT-001.md
- docs/07-ui-ux/UI-TENANT-001-screens.md
- docs/10-testing/TEST-TENANT-001-test-plan.md

9. Branch / Office Hierarchy and Scoped Administration

Phase: S1-S4 foundation | Schema: org plus security | Approved source key: BRANCH

Approved conclusion

Branch and office hierarchy lets one tenant operate many offices, branches, sites, or regional units while preserving scoped administration. A complete tenant admin can see the whole tenant, while branch admins and branch-scoped users can only see or act inside their approved branch scope.

Approved delivery

- Tenant branch and office tree with parent-child hierarchy, effective dates, status, timezone, country, state, and address metadata.
- Scoped administration model for tenant-wide admins, branch admins, HR operators, payroll operators, managers, and employees.
- Employee branch assignment with effective dating so historical reports and payroll remain correct.
- Branch-aware RBAC and ABAC enforcement across Core HR, Leave, Attendance, Payroll, Reports, exports, jobs, and events.
- Branch hierarchy events and audit evidence for branch changes, scope assignment, and employee branch movement.

Architecture and dependencies

- Branch hierarchy is modeled as tenant-owned organization data and exposed through shared scope resolvers.
- Authorization combines tenant context, RBAC permissions, ABAC attributes, and branch scope.
- Branch-scoped users cannot access sibling branch data unless a higher scope is explicitly granted.
- Downstream modules consume branch assignment through Core HR APIs/events and do not duplicate branch ownership logic.

Database model

Key tables / records
org.BranchOffice
org.BranchOfficeHierarchyClosure
org.EmployeeBranchAssignment
security.BranchScopeAssignment
security.BranchScopeDecisionLog

API and OpenAPI examples

- GET /api/v1/branches
- POST /api/v1/branches
- GET /api/v1/branches/{id}/tree
- POST /api/v1/branches/{id}/scope-assignments
- GET /api/v1/security/branch-scopes

Event examples

- BranchCreated
- BranchUpdated
- BranchHierarchyChanged
- BranchScopeAssigned
- BranchScopeRevoked
- EmployeeBranchAssignmentChanged

Example scenario

Tenant A has a head office, Mumbai branch, Delhi branch, and Pune branch. A tenant admin can review all offices. The Mumbai branch admin can manage only Mumbai employees, approvals, attendance, reports, and exports. They cannot open Delhi employee details or payroll reports.

Test and quality focus

- Branch admin cannot read, export, approve, or update sibling branch data.
- Tenant admin can view all branches inside the same tenant but never another tenant.
- Historical employee branch assignment works for as-of reports, payroll evidence, and audit.

Source document paths

- docs/02-product-requirements/FEAT-BRANCH-001-business-requirements.md
- docs/09-development/TECH-BRANCH-001-technical-design.md
- docs/06-database/DB-DESIGN-BRANCH-001.md
- docs/07-ui-ux/UI-BRANCH-001-screens.md
- docs/10-testing/TEST-BRANCH-001-test-plan.md

10. Identity, Authentication, RBAC, and ABAC

Phase: S1 foundation | Schema: security | Approved source key: IDENTITY

Approved conclusion

Identity is the central security module. It handles authentication, sessions, roles, permissions, policies, delegation, impersonation, emergency access, and authorization decision traceability. The approved design combines RBAC for role permissions and ABAC for contextual rules.

Approved delivery

- Local login, OIDC/SSO, MFA, adaptive risk checks, and refresh token rotation.
- RBAC permissions, ABAC policies, deny-by-default authorization, and policy decision tracing.
- SCIM user and group provisioning for enterprise identity integration.
- Device/session management, global logout, remote session termination, and distributed revocation.
- Break Glass emergency access with reason, expiry, approval option, revocation, and audit.

Architecture and dependencies

- Client requests pass through API Gateway, Authentication Service, Token Service, PEP, and PDP.
- The PEP enforces permissions at API, handler, job, report, and UI capability boundaries.
- The PDP evaluates role, resource, tenant, user, delegation, impersonation, and ABAC policy context.
- Distributed session and revocation state keep multiple app instances consistent.

Database model

Key tables / records
security.UserAccount
security.Role
security.Permission
security.RolePermission
security.UserRole
security.AbacPolicy
security.Session
security.RefreshToken
security.Delegation
security.PasswordPolicy
security.PasswordHistory
security.AuthSecurityEvent
security.BreakGlassAccess
security.BreakGlassSession

API and OpenAPI examples

- POST /api/v1/auth/login
- POST /api/v1/auth/sso/start
- POST /api/v1/auth/mfa/verify
- POST /api/v1/auth/refresh
- GET /api/v1/security/sessions
- POST /api/v1/security/break-glass

- GET /api/v1/security/authorization-trace/{correlationId}

Event examples

- UserLoggedIn
- MfaChallengeCompleted
- SessionRevoked
- RoleAssignmentChanged
- TenantRoleMatrixChanged
- BreakGlassActivated
- AuthSecurityEventRecorded

Example scenario

A manager can approve a leave request only when RBAC grants Leave.Approve and ABAC confirms the employee belongs to the manager's allowed team, legal entity, or delegated scope. The decision record stores tenant, user, permission, policy version, result, reason, and correlation id.

Test and quality focus

- Unauthorized users cannot access secured endpoints or emergency access.
- Refresh token reuse is detected and revokes the session family.
- Distributed logout and role change invalidation work across all app instances.
- Brute-force, credential stuffing, adaptive MFA, and lockout controls are verified.

Source document paths

- docs/02-product-requirements/FEAT-IDENTITY-001-business-requirements.md
- docs/09-development/TECH-IDENTITY-001-technical-design.md
- docs/06-database/DB-DESIGN-IDENTITY-001.md
- docs/07-ui-ux/UI-IDENTITY-001-screens.md
- docs/10-testing/TEST-IDENTITY-001-test-plan.md

11. Effective Dating and Bitemporal Core

Phase: S2 foundation | Schema: core and temporal history | Approved source key: EFFECTIVE-DATING

Approved conclusion

Effective dating is a platform pattern for any business fact that changes over time. It allows the system to answer what was true on a business date, what the system knew at a system time, and why a change happened. This is essential for payroll, assignments, policies, workflow versions, compliance, and reporting.

Approved delivery

- Shared effective dating service, period validator, as-of query service, change request flow, and history explanation.
- Mandatory valid-time columns for time-changing facts.
- SQL Server temporal table support for high-value history plus approved fallback patterns.
- Backdated, current, future-dated, supersede, cancel, and bulk effective-dated operations.
- Event publication through the outbox for every approved effective-dated change.

Architecture and dependencies

- Business APIs call application services, which call the Effective Dating Service before repositories.
- The Period Validator prevents invalid periods, overlaps, duplicate open-ended rows, and unsafe backdating.
- As-of reads are consistent across API, reporting, rules, workflow, and payroll.
- Sensitive corrections can be routed through Workflow Studio before activation.

Database model

Key tables / records
core.EffectiveDatedEntityRegistration
core.EffectiveDatedChangeRequest
core.EffectivePeriodConflict
EffectiveFrom
EffectiveTo
EffectiveStatus
ChangeReason
ApprovalReferenceId
SupersededById
VersionNumber or rowversion

API and OpenAPI examples

- GET /api/v1/{module}/{resource}?asOfDate=YYYY-MM-DD
- POST /api/v1/{module}/{resource}/future-change
- POST /api/v1/{module}/{resource}/backdated-correction
- GET /api/v1/{module}/{resource}/{id}/history/explain

Event examples

- EffectiveDatedRecordChanged
- EffectiveDatedChangeApproved
- EffectiveDatedChangeRejected
- EffectiveDatedConflictDetected

Example scenario

If an employee's department changes from Sales to Finance effective 1 July, reports for 30 June still show Sales, payroll from July uses Finance, and the history screen explains who approved the change, when it was entered, and which old row was superseded.

Test and quality focus

- Overlap prevention works for employee assignment, salary assignment, policy version, and workflow/rule versions.
- As-of API, report, and payroll results agree for the same business date.
- Backdated changes produce audit entries, outbox events, and explain-history evidence.

Source document paths

- docs/02-product-requirements/FEAT-EFFECTIVE-DATING-001-business-requirements.md
- docs/09-development/TECH-EFFECTIVE-DATING-001-technical-design.md
- docs/06-database/DB-DESIGN-EFFECTIVE-DATING-001.md
- docs/07-ui-ux/UI-EFFECTIVE-DATING-001-screens.md
- docs/10-testing/TEST-EFFECTIVE-DATING-001-test-plan.md

12. Audit and Time Machine

Phase: S2 foundation | Schema: audit | Approved source key: AUDIT

Approved conclusion

Audit and Time Machine provide evidence. Every important change, authorization decision, security event, workflow decision, export, and sensitive read is traceable. Time Machine reconstructs business state using effective dating, temporal history, audit records, workflow evidence, and events.

Approved delivery

- Shared audit capture services consumed by all Phase 7A modules.
- Append-only audit records and field changes with masking, classification, retention, and legal hold.
- Security event collection for login, MFA, ABAC, emergency access, and suspicious activity.
- Time-machine snapshots and explain-history views.
- Controlled audit exports with purpose, approval, expiry, and access tracking.

Architecture and dependencies

- Modules write audit through shared middleware, domain-event handlers, and approved capture points.
- Modules do not create private audit tables.
- High-risk audit classes can use hash chaining or ledger-like tamper evidence.
- Audit reads are themselves audited.

Database model

Key tables / records
audit.AuditRecord
audit.AuditFieldChange
audit.SecurityEvent
audit.TimeMachineSnapshot
audit.AuditExportRequest

API and OpenAPI examples

- GET /api/v1/audit/search
- GET /api/v1/audit/entities/{entityName}/{entityId}/history
- GET /api/v1/audit/security-events
- POST /api/v1/audit/exports
- POST /api/v1/audit/time-machine/reconstruct

Event examples

- AuditRecordCreated
- SecurityEventRecorded
- TimeMachineSnapshotGenerated
- AuditExportRequested
- AuditExportDownloaded

Example scenario

When payroll is published, the audit trail can show the run, source snapshot hash, rule versions, approver, exceptions, generated payslips, and every user who downloaded payroll evidence.

Test and quality focus

- Sensitive values are masked or encrypted and passwords/secrets are never stored.
- Entity history lookup is tenant-filtered and performant.
- Legal hold blocks purge and exports expire correctly.

Source document paths

- docs/02-product-requirements/FEAT-AUDIT-001-business-requirements.md
- docs/09-development/TECH-AUDIT-001-technical-design.md
- docs/06-database/DB-DESIGN-AUDIT-001.md
- docs/07-ui-ux/UI-AUDIT-001-screens.md
- docs/10-testing/TEST-AUDIT-001-test-plan.md

13. Event Bus and Transactional Outbox

Phase: S2 foundation | Schema: integration | Approved source key: EVENT-BUS

Approved conclusion

The event backbone allows modules to communicate without direct database coupling. Phase 7A uses RabbitMQ first behind an event bus abstraction, with Azure Service Bus available later through adapter replacement.

Approved delivery

- Event contract registry with versioned schemas and compatibility status.
- Transactional outbox so business change and event creation are atomic.
- Outbox publisher worker with retry, backoff, dead-letter handling, and replay approval.
- Inbox store for idempotent consumers.
- Tenant-scoped event metadata, classification, correlation, and causation ids.

Architecture and dependencies

- A module command writes domain data and OutboxEvent in the same database transaction.
- The publisher worker sends events to RabbitMQ through IEventBus.
- Consumers record InboxMessage before side effects to prevent duplicate processing.
- Event contracts are versioned; renaming or removing events needs deprecation planning.

Database model

Key tables / records
integration.EventContract
integration.OutboxEvent
integration.InboxMessage
integration.DeadLetterEvent
integration.EventReplayRequest

API and OpenAPI examples

- GET /api/v1/integration/events/contracts
- GET /api/v1/integration/outbox
- GET /api/v1/integration/dead-letter
- POST /api/v1/integration/events/replay

Event examples

- EmployeeCreated
- EmployeeAssignmentChanged
- LeaveApproved
- AttendanceSummaryApproved
- PayrollRunPublished
- ReportExportCreated

Example scenario

When LeaveApproved is committed, the outbox event is committed with it. Payroll can consume the event later to update loss-of-pay input. If RabbitMQ is temporarily down, the outbox keeps the event pending until the worker retries.

Test and quality focus

- Business data and outbox event are atomic.
- Duplicate events are rejected by EventId and duplicate consumer work is blocked by InboxMessage.
- Dead-letter replay requires approval and preserves reason and audit trail.

Source document paths

- docs/02-product-requirements/FEAT-EVENT-BUS-001-business-requirements.md
- docs/09-development/TECH-EVENT-BUS-001-technical-design.md
- docs/06-database/DB-DESIGN-EVENT-BUS-001.md
- docs/07-ui-ux/UI-EVENT-BUS-001-screens.md
- docs/10-testing/TEST-EVENT-BUS-001-test-plan.md

14. Rule Engine

Phase: S3 foundation | Schema: rules | Approved source key: RULE-ENGINE

Approved conclusion

The Rule Engine prevents hardcoded business rules. Leave eligibility, attendance penalties, workflow routing, payroll formulas, statutory rules, feature flag evaluation, and compliance conditions are represented as versioned, effective-dated, testable configuration.

Approved delivery

- Rule set and rule version lifecycle: draft, validate, simulate, approve, publish, rollback, deprecate.
- Decision tables, expression rules, JSON schema validation, and safe deterministic evaluation.
- Simulation workspace with explanations before publish.
- Runtime evaluation service for modules with tenant, version, correlation, and trace metadata.
- Evaluation logs with hashes and summaries for audit and troubleshooting.

Architecture and dependencies

- Modules call the shared Rule Evaluation API; they do not embed business thresholds in code.
- Rule Resolver selects the effective published version for tenant, module, date, and context.
- Evaluator returns outputs plus explanation metadata.
- Payroll and compliance rule versions are retained while referenced by payroll runs.

Database model

Key tables / records
rules.RuleSet
rules.RuleSetVersion
rules.DecisionTable
rules.RuleSimulation
rules.RuleEvaluationLog

API and OpenAPI examples

- POST /api/v1/rules/rule-sets
- POST /api/v1/rules/{ruleSetId}/versions/{versionId}/simulate
- POST /api/v1/rules/{ruleSetId}/versions/{versionId}/submit
- POST /api/v1/rules/{ruleSetId}/versions/{versionId}/publish
- GET /api/v1/rules/evaluations/{correlationId}

Event examples

- RuleSetVersionPublished
- RuleSetVersionDeprecated
- RuleSimulationCompleted
- RuleEvaluationFailed

Example scenario

Instead of coding 'leave days greater than 5 needs HR approval', the tenant config can define a rule: if leave type is annual and duration exceeds the configured threshold, route to HR after manager approval. Changing the threshold is a rule version change, not a code change.

Test and quality focus

- Rule simulation produces deterministic outputs and explanations.
- Invalid JSON/schema payloads cannot be published.
- Runtime modules evaluate the intended effective version for date and tenant.

Source document paths

- docs/02-product-requirements/FEAT-RULE-ENGINE-001-business-requirements.md
- docs/09-development/TECH-RULE-ENGINE-001-technical-design.md
- docs/06-database/DB-DESIGN-RULE-ENGINE-001.md
- docs/07-ui-ux/UI-RULE-ENGINE-001-screens.md
- docs/10-testing/TEST-RULE-ENGINE-001-test-plan.md

15. Workflow Studio

Phase: S3 foundation and S9 hardening | Schema: workflow | Approved source key: WORKFLOW

Approved conclusion

Workflow Studio is the approval and process engine for Phase 7A and future modules. It handles routed approvals, tasks, delegation, escalation, SLA timers, comments, decisions, versioned definitions, and outcome events.

Approved delivery

- Workflow definition and published version lifecycle.
- Runtime workflow instances pinned to exact published version.
- Task inbox, decisions, delegation, reassignment, escalation, and SLA events.
- Workflow UI for designers, reviewers, task owners, and auditors.
- Outcome events that business modules can consume without direct coupling.

Architecture and dependencies

- Modules request workflow start and receive task/outcome callbacks or events.
- Workflow uses Rule Engine for routing, eligibility, and dynamic approver decisions.
- Workflow history links back to the source business entity.
- Published definitions are immutable; running instances do not change silently when a new version is published.

Database model

Key tables / records
workflow.WorkflowDefinition
workflow.WorkflowDefinitionVersion
workflow.WorkflowInstance
workflow.WorkflowTask
workflow.WorkflowDecision
workflow.WorkflowSlaEvent

API and OpenAPI examples

- POST /api/v1/workflows/definitions
- POST /api/v1/workflows/definitions/{id}/publish
- POST /api/v1/workflows/instances
- GET /api/v1/workflows/tasks
- POST /api/v1/workflows/tasks/{taskId}/decision

Event examples

- WorkflowInstanceStarted
- WorkflowTaskAssigned
- WorkflowTaskDecided
- WorkflowSlaBreached
- WorkflowInstanceCompleted

Example scenario

A payroll structure change can require maker-checker approval. Workflow creates a task for Payroll Reviewer, uses SLA rules for reminders, records the decision, and only then allows the salary structure version to be published.

Test and quality focus

- Running instance remains pinned to original workflow version.
- Delegation and escalation create correct task ownership and audit evidence.
- Task inbox respects assignee, role, delegate, tenant, and ABAC filters.

Source document paths

- docs/02-product-requirements/FEAT-WORKFLOW-001-business-requirements.md
- docs/09-development/TECH-WORKFLOW-001-technical-design.md
- docs/06-database/DB-DESIGN-WORKFLOW-001.md
- docs/07-ui-ux/UI-WORKFLOW-001-screens.md
- docs/10-testing/TEST-WORKFLOW-001-test-plan.md

16. Configuration-as-Data

Phase: S3 foundation | Schema: config | Approved source key: CONFIGURATION

Approved conclusion

Configuration-as-Data makes tenant behavior changeable without core code edits. Forms, feature flags, module manifests, policy payloads, provider settings, UI slots, navigation, rules, and workflow references become governed data with schema validation and published versions.

Approved delivery

- Module manifest registry for dependencies, features, APIs, events, and UI extension slots.
- Configuration schema registry using JSON Schema.
- Versioned configuration items with effective dates, approval, publish, rollback, export, and import.
- Tenant-aware feature flags aligned to OpenFeature concepts.
- Promotion path between sandbox and production with traceability.

Architecture and dependencies

- Runtime services resolve the effective published configuration through shared providers.
- Secrets are references to approved secret storage, never payload values.
- Configuration publish invalidates cache and emits events.
- Customer customization happens through configuration, feature flags, extensions, and plugins.

Database model

Key tables / records
config.ModuleManifest
config.ConfigurationSchema
config.ConfigurationItem
config.ConfigurationVersion
config.FeatureFlag
config.ConfigurationImportExport

API and OpenAPI examples

- GET /api/v1/configuration/schemas
- POST /api/v1/configuration/items
- POST /api/v1/configuration/versions/{id}/validate
- POST /api/v1/configuration/versions/{id}/publish
- POST /api/v1/configuration/import-export

Event examples

- ConfigurationVersionPublished
- ConfigurationRollbackCompleted
- FeatureFlagChanged
- ModuleManifestRegistered
- ConfigurationImportCompleted

Example scenario

A tenant can add a new onboarding form field or change leave policy display text by publishing a validated configuration version. Core HRMS code does not change.

Test and quality focus

- Published versions are immutable and schema-validated.
- Feature flags are tenant-scoped and audited.
- Import/export has approval, rollback, and no secret leakage.

Source document paths

- docs/02-product-requirements/FEAT-CONFIGURATION-001-business-requirements.md
- docs/09-development/TECH-CONFIGURATION-001-technical-design.md
- docs/06-database/DB-DESIGN-CONFIGURATION-001.md
- docs/07-ui-ux/UI-CONFIGURATION-001-screens.md
- docs/10-testing/TEST-CONFIGURATION-001-test-plan.md

17. Core HR and Employee Self-Service

Phase: S4 business module | Schema: hr | Approved source key: CORE-HR

Approved conclusion

Core HR is the employee master data module. It owns person, employee, assignment, organization, location, document metadata, and self-service change requests. Leave, attendance, payroll, workflow, rules, reports, and integrations consume Core HR through APIs/events, not direct ownership.

Approved delivery

- Employee and person records with protected personal information.
- Effective-dated assignments, manager hierarchy, departments, designations, grades, locations, and legal entities.
- Employee self-service change requests routed through Workflow Studio.
- Employee document metadata with secure file references.
- Events for employee creation, update, assignment change, and lifecycle status.

Architecture and dependencies

- Core HR APIs call the Effective Dating Service for assignment and org history.
- Manager hierarchy queries use ABAC and hierarchy service, not unrestricted employee reads.
- Other modules consume Core HR APIs/events and do not duplicate master data.
- Search/report projections can be rebuilt from Core HR source plus events.

Database model

Key tables / records
hr.Person
hr.Employee
hr.EmployeeAssignment
hr.OrganizationUnit
hr.Location
hr.EssChangeRequest
hr.EmployeeDocumentMetadata

API and OpenAPI examples

- GET /api/v1/hr/employees
- GET /api/v1/hr/employees/{id}?asOfDate=YYYY-MM-DD
- POST /api/v1/hr/ess/change-requests
- GET /api/v1/hr/organization-units
- GET /api/v1/hr/locations

Event examples

- EmployeeCreated
- EmployeeUpdated
- EmployeeAssignmentChanged
- EmployeeStatusChanged
- EssChangeRequestApproved

Example scenario

When an employee transfers to a new manager from next month, Core HR creates an effective-dated assignment change. Leave approvals before that date still route to the old manager; approvals after that date route to the new manager.

Test and quality focus

- Employee number is unique per tenant.
- Manager hierarchy and as-of assignment queries return correct historical state.
- Sensitive personal fields are encrypted, masked, and access controlled.

Source document paths

- docs/02-product-requirements/FEAT-CORE-HR-001-business-requirements.md
- docs/09-development/TECH-CORE-HR-001-technical-design.md
- docs/06-database/DB-DESIGN-CORE-HR-001.md
- docs/07-ui-ux/UI-CORE-HR-001-screens.md
- docs/10-testing/TEST-CORE-HR-001-test-plan.md

18. Leave Management

Phase: S5 business module | Schema: leave | Approved source key: LEAVE

Approved conclusion

Leave Management delivers the first end-to-end HR business workflow: policies, balances, applications, approvals, cancellation, adjustment, holiday calendars, payroll impact, and audit. It proves the platform foundations work together.

Approved delivery

- Tenant-specific leave types, policy versions, eligibility, accrual, day-count, carry-forward, and approval workflow references.
- Leave request lifecycle: apply, validate, reserve, approve, reject, withdraw, cancel, and correct.
- Ledger-based leave transactions with rebuildable balance projections.
- Holiday calendars scoped by tenant and location.
- Payroll-impact events for unpaid leave, corrections, and balance-affecting decisions.

Architecture and dependencies

- Leave consumes Tenant, Identity, Effective Dating, Audit, Event Bus, Rule Engine, Workflow, Configuration, and Core HR.
- Balances are derived from immutable transactions, not manually overwritten totals.
- Rule Engine handles eligibility, accrual, day count, and carry-forward logic.
- Workflow handles approval routing; Event Bus publishes payroll and reporting impacts.

Database model

Key tables / records
leave.LeaveType
leave.LeavePolicyVersion
leave.HolidayCalendar
leave.Holiday
leave.LeaveRequest
leave.LeaveTransaction
leave.LeaveBalanceProjection

API and OpenAPI examples

- GET /api/v1/leave/types
- GET /api/v1/leave/balances
- POST /api/v1/leave/requests
- POST /api/v1/leave/requests/{id}/withdraw
- GET /api/v1/leave/requests/{id}/history
- POST /api/v1/leave/policies/{id}/preview

Event examples

- LeaveRequested
- LeaveApproved
- LeaveRejected
- LeaveCancelled
- LeaveBalanceChanged
- LeavePayrollImpactReady

Example scenario

An employee applies for 3 days annual leave. The system checks eligibility and calendar, reserves 3 days in the ledger, starts workflow, and after approval debits the ledger. If the request is cancelled before payroll cutoff, a reversing ledger transaction and payroll-impact event are created.

Test and quality focus

- Duplicate requests cannot double-reserve or double-debit balance.
- Ledger can rebuild projections exactly.
- Policy versions support historical and future-dated rules.

Source document paths

- docs/02-product-requirements/FEAT-LEAVE-001-business-requirements.md
- docs/09-development/TECH-LEAVE-001-technical-design.md
- docs/06-database/DB-DESIGN-LEAVE-001.md
- docs/07-ui-ux/UI-LEAVE-001-screens.md
- docs/10-testing/TEST-LEAVE-001-test-plan.md

19. Attendance and First Connector

Phase: S6 business module | Schema: attendance | Approved source key: ATTENDANCE

Approved conclusion

Attendance captures raw punches, reconciles daily attendance summaries, supports regularization requests, and feeds payroll. The first connector framework proves how future devices or providers can be added through adapters and configuration.

Approved delivery

- Attendance policy versions linked to rules and calendars.
- Device and connector configuration with secret references.
- Raw punch ingestion with deduplication and immutability.
- Daily summary calculation for present, absent, late, early leave, work minutes, and payroll impact.
- Regularization workflow for employee corrections and manager/HR approval.

Architecture and dependencies

- Connector/Web/API input flows into Punch Ingestion, Raw Punch Store, Attendance Engine, Rules, and Payroll Events.
- Raw punches are immutable; corrections are approved regularization records.
- Daily summaries are rebuildable from raw punches plus approved adjustments.
- Connector adapters are provider-specific; business logic stays provider-neutral.

Database model

Key tables / records
attendance.AttendancePolicyVersion
attendance.AttendanceDevice
attendance.RawPunch
attendance.AttendanceDaySummary
attendance.RegularizationRequest
attendance.ConnectorSyncRun

API and OpenAPI examples

- POST /api/v1/attendance/punches
- GET /api/v1/attendance/summaries
- POST /api/v1/attendance/regularization-requests
- GET /api/v1/attendance/connectors/sync-runs
- POST /api/v1/attendance/reconciliation/run

Event examples

- RawPunchReceived
- AttendanceSummaryCalculated
- RegularizationRequested
- RegularizationApproved
- AttendancePayrollImpactReady
- ConnectorSyncCompleted

Example scenario

A biometric device sends the same punch twice. The unique external punch key marks the duplicate without double-counting work time. If the employee forgot checkout, they submit regularization, workflow approves it, and the daily summary is recalculated for payroll.

Test and quality focus

- Duplicate external punches are detected and do not double count.
- Summary rebuild produces consistent payroll-impact results.
- Connector failures and sync gaps are auditable and recoverable.

Source document paths

- docs/02-product-requirements/FEAT-ATTENDANCE-001-business-requirements.md
- docs/09-development/TECH-ATTENDANCE-001-technical-design.md
- docs/06-database/DB-DESIGN-ATTENDANCE-001.md
- docs/07-ui-ux/UI-ATTENDANCE-001-screens.md
- docs/10-testing/TEST-ATTENDANCE-001-test-plan.md

20. Shift Foundation

Phase: S6-S8 foundation | Schema: attendance | Approved source key: SHIFT-FOUNDATION

Approved conclusion

Shift Foundation defines basic shift templates, employee shift assignments, exceptions, and shift resolution. It is intentionally included in Phase 7A because attendance and payroll cannot be reliable without knowing which shift applied to an employee on a date. Advanced roster planning, workforce demand planning, and shift swaps remain later-phase work.

Approved delivery

- Configurable shift definitions with start/end time, grace rules, break handling, night shift flag, timezone, and effective versions.
- Effective-dated employee shift assignments linked to branch, department, role, or employee where permitted.
- Controlled one-day or temporary shift overrides routed through workflow when tenant policy requires approval.
- Shift resolver service used by attendance summary calculation, regularization, reports, and payroll source snapshots.
- Audit, events, and explainability for which shift was used in a daily attendance or payroll result.

Architecture and dependencies

- Attendance asks the Shift Resolver for the effective shift before calculating late, early, absent, night, overtime, or payroll-impact values.
- Shift definitions are versioned and effective-dated so past attendance and payroll results remain reproducible.
- Branch and role defaults can be overridden by employee-specific assignments when allowed by tenant policy.
- Payroll consumes shift-aware attendance summaries, not raw shift rules, so published payroll stays explainable.

Database model

Key tables / records
attendance.ShiftDefinition
attendance.ShiftDefinitionVersion
attendance.EmployeeShiftAssignment
attendance.EmployeeShiftOverride
attendance.ShiftResolutionLog

API and OpenAPI examples

- GET /api/v1/attendance/shifts
- POST /api/v1/attendance/shifts
- POST /api/v1/attendance/shifts/{id}/publish
- POST /api/v1/attendance/shift-assignments
- POST /api/v1/attendance/shift-overrides
- GET /api/v1/attendance/employees/{employeeId}/shift?date=YYYY-MM-DD

Event examples

- ShiftDefinitionPublished
- ShiftDefinitionRetired
- EmployeeShiftAssigned
- EmployeeShiftAssignmentChanged
- ShiftOverrideApproved
- ShiftPayrollImpactChanged

Example scenario

An employee is assigned to a 10:00 to 19:00 shift for July. If they punch in at 10:20, attendance applies that shift's grace rule. If the employee is temporarily assigned to a night shift for one day, the approved override is used only for that date and payroll can explain the result.

Test and quality focus

- Effective shift resolution chooses employee override, employee assignment, branch default, and tenant default in the approved priority order.
- Overnight shifts calculate work date, late arrival, early exit, and payable minutes correctly.
- Payroll snapshots reference the shift-aware attendance summary and remain reproducible after shift rules change.

Source document paths

- docs/02-product-requirements/FEAT-SHIFT-FOUNDATION-001-business-requirements.md
- docs/09-development/TECH-SHIFT-FOUNDATION-001-technical-design.md
- docs/06-database/DB-DESIGN-SHIFT-FOUNDATION-001.md
- docs/07-ui-ux/UI-SHIFT-FOUNDATION-001-screens.md
- docs/10-testing/TEST-SHIFT-FOUNDATION-001-test-plan.md

21. Payroll and India Compliance

Phase: S7-S8 business module | Schema: payroll | Approved source key: PAYROLL

Approved conclusion

Payroll is the highest-risk Phase 7A business module. It calculates employee payroll from Core HR, Leave, Attendance, salary structures, declarations, and effective-dated statutory rules. Published payroll must be reproducible, explainable, immutable, and compliant with the India-first foundation.

Approved delivery

- Payroll calendars, periods, cutoff, dry run, approval, lock, publish, and correction flow.
- Salary components, salary structure versions, and effective-dated employee salary assignments.
- Source snapshot for Core HR, Leave, Attendance, and component inputs.
- Calculation engine for earnings, deductions, arrears, LOP, statutory components, and exceptions.
- India compliance foundation: PF, ESI, PT, LWF, TDS, Form 16, FBP, and revisions where applicable.
- Payslip generation and employee access controls.

Architecture and dependencies

- Payroll consumes Core HR, Leave, Attendance, Rule Engine, Workflow, Audit, and Event Bus.
- The run captures source snapshot hash and rule-version snapshot before calculation.
- Rule Engine evaluates salary, eligibility, statutory, and tax logic.
- Workflow approves sensitive configuration and payroll run publish.
- Published runs are immutable; corrections create adjustment runs or controlled correction records.

Database model

Key tables / records
payroll.PayrollCalendar
payroll.PayPeriod
payroll.SalaryComponent
payroll.SalaryStructureVersion
payroll.EmployeeSalaryAssignment
payroll.PayrollRun
payroll.PayrollRunEmployee
payroll.PayrollRunComponent
payroll.StatutoryRuleVersion
payroll.Payslip

API and OpenAPI examples

- POST /api/v1/payroll/runs/dry-run
- POST /api/v1/payroll/runs/{id}/approve
- POST /api/v1/payroll/runs/{id}/publish
- GET /api/v1/payroll/runs/{id}/employees/{employeeid}/explain
- GET /api/v1/payroll/payslips
- POST /api/v1/payroll/statutory-rule-versions

Event examples

- PayrollRunStarted
- PayrollRunCalculated
- PayrollRunExceptionRaised
- PayrollRunApproved
- PayrollRunPublished
- PayslipPublished

Example scenario

For June payroll, the run snapshots employee assignments, approved leave loss-of-pay, attendance summaries, salary assignments, declarations, and statutory rule versions. A future salary change effective July does not change June results. After publish, June payroll is locked; corrections go through a revision or adjustment process.

Test and quality focus

- Published payroll can be reproduced from source and rule snapshots.
- Unauthorized users cannot see payroll data outside legal entity, payroll group, or employee-self scope.
- Statutory rule versions are effective-dated and cannot be purged while referenced.
- Explain-calculation returns lineage for each component.

Source document paths

- docs/02-product-requirements/FEAT-PAYROLL-001-business-requirements.md
- docs/09-development/TECH-PAYROLL-001-technical-design.md
- docs/06-database/DB-DESIGN-PAYROLL-001.md
- docs/07-ui-ux/UI-PAYROLL-001-screens.md
- docs/10-testing/TEST-PAYROLL-001-test-plan.md

22. Standard Reports

Phase: S10 business module | Schema: reporting | Approved source key: REPORTING

Approved conclusion

Standard Reports provide the first governed reporting layer. Reports are versioned, permissioned, tenant-scoped, export-controlled, and supported by rebuildable projections. Reporting uses approved APIs, events, and read models rather than unsafe direct cross-module coupling.

Approved delivery

- Report catalog with definitions, parameters, classifications, permissions, and versions.
- Report run and export flow with status, purpose, expiry, and audit.
- Employee and payroll projections for common operational/statutory reports.
- As-of reporting using Effective Dating.
- Export controls for sensitive data, approvals where required, download limits, and expiration.

Architecture and dependencies

- Module events and approved APIs feed reporting projections.
- Report Query Service applies tenant, RBAC, ABAC, filters, pagination, and masking.
- Export Service is asynchronous and audited.
- Projection Builder can rebuild read models from source modules/events.

Database model

Key tables / records
reporting.ReportDefinition
reporting.ReportDefinitionVersion
reporting.ReportRun
reporting.ReportExport
reporting.EmployeeReportProjection
reporting.PayrollReportProjection

API and OpenAPI examples

- GET /api/v1/reports/catalog
- POST /api/v1/reports/{reportCode}/run
- GET /api/v1/reports/runs/{runId}
- POST /api/v1/reports/runs/{runId}/export
- GET /api/v1/reports/exports/{exportId}/download

Event examples

- ReportRunRequested
- ReportRunCompleted
- ReportExportCreated
- ReportExportDownloaded
- ReportProjectionRebuilt

Example scenario

A payroll summary report uses the payroll projection linked to the exact payroll run and rule snapshot. An HR admin with the correct legal-entity scope can export it with a purpose and expiry; the export download is audited.

Test and quality focus

- Report results are tenant-scoped, paginated, masked, and permission-filtered.
- As-of report dates match Effective Dating service results.
- Exports expire and access logs are complete.

Source document paths

- docs/02-product-requirements/FEAT-REPORTING-001-business-requirements.md
- docs/09-development/TECH-REPORTING-001-technical-design.md
- docs/06-database/DB-DESIGN-REPORTING-001.md
- docs/07-ui-ux/UI-REPORTING-001-screens.md
- docs/10-testing/TEST-REPORTING-001-test-plan.md

23. Security, Quality, and Hardening Rules

- Every API must be versioned, documented in OpenAPI, tested, and protected by tenant, RBAC, ABAC, and audit controls.
- Unit test coverage target is at least 85 percent, with higher scrutiny for payroll, security, RLS, rules, workflow, effective dating, and audit.
- Integration tests must verify cross-module flows such as leave to payroll, attendance to payroll, workflow outcome to business state, and event retries.
- End-to-end tests must cover user workflows for employee, manager, HR admin, payroll, tenant admin, and auditor roles.
- Security tests must cover cross-tenant abuse, privilege escalation, IDOR, brute force, session misuse, export misuse, and sensitive data masking.
- Performance tests must cover tenant lookup, RLS queries, rule evaluation, workflow inbox, event lag, report exports, and payroll runs.
- Runbooks are required for provider outage, database failover, event backlog, dead-letter replay, payroll publish issue, export incident, and tenant suspension.

24. What Is Not Part of Phase 7A

- Recruitment, applicant tracking, advanced onboarding, performance, LMS, engagement, expenses, assets, service desk, advanced BI, advanced roster planning, shift swaps, workforce demand planning, and marketplace are later modules.
- Multi-country payroll packs are later; Phase 7A establishes India-first payroll and compliance foundation.
- Advanced autonomous AI agents and AI-driven HR decisions are not part of Phase 7A business module delivery.
- Customer-specific code changes are not allowed; customization must use configuration, extensions, feature flags, providers, or plugins.

25. How AI Should Use This PDF

- Treat this as a consolidated approved Phase 7A reference, not as raw implementation code.
- When answering about scope, use the 15 module names in this document.
- When answering about database, mention schemas, TenantId, RLS, effective dating, audit, and immutability where relevant.
- When answering about architecture, explain shared platform services before business modules.
- When answering about customization, say configuration, feature flags, extensions, providers, and plugins are preferred; core code changes are prohibited for customer-specific behavior.
- When answering about sequencing, say foundations come before business modules and phase gates cannot be skipped.
- Do not use old launch-scope language. Use Phase 7A, later phases, or approved phase terminology.
- If asked for exact legal wording, source paths, or full requirements, refer back to the approved source documents.

26. External and Web References

- Microsoft SQL Server Row-Level Security - <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security>
- Microsoft SQL Server Temporal Tables - <https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables>
- Microsoft SQL Server Ledger - <https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ledger-overview>
- OpenAPI Specification - <https://spec.openapis.org/oas/latest.html>
- CloudEvents - <https://cloudevents.io/>
- RabbitMQ Reliability Guide - <https://www.rabbitmq.com/docs/reliability>
- JSON Schema - <https://json-schema.org/>
- OpenFeature Specification - <https://openfeature.dev/specification/>
- OWASP API Security Top 10 - <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

- WCAG 2.2 - <https://www.w3.org/TR/WCAG22/>
- EPFO India - https://www.epfindia.gov.in/site_en/index.php
- Income Tax Department India - <https://www.incometax.gov.in/iec/foportal/>
- Web visual: Wikimedia Commons team meeting photo - WLM international team meeting Vienna 2023-05-27
- Web visual: Wikimedia Commons server infrastructure photo - Wikimedia Servers-0051 19